

PHIDATA + GROQ

Complete Agent Development Guide

Beginner to Advanced - 25 Chapters - Production Ready

AI Agents	Multi-Agent Systems	RAG Systems
Autonomous Workflows	AI SaaS Products	Research Agents
Coding Agents	Voice Agents	Production Deployment
FastAPI Integration	Streamlit Frontends	Docker Deployment

25 Chapters - Production-Grade Examples - Real Projects

CHAPTER 1

Understanding AI Agents

An **AI Agent** is an intelligent software system capable of reasoning, planning, using tools, accessing memory, interacting with APIs, collaborating with other agents, and solving tasks autonomously.

- Reasoning and Planning
- Tool Usage
- Memory Access
- API Interaction
- Multi-Agent Collaboration
- Autonomous Task Solving

Traditional AI vs AI Agent

Traditional AI:

```
User -> AI -> Response
```

AI Agent:

```
User
| Planning
| Reasoning
| Tool Usage
| Memory Access
| Validation
| Final Output
```

CHAPTER 2

What is Phidata?

Phidata is a powerful framework for building production AI systems. It handles all the heavy lifting so you can focus on business logic.

What Phidata Handles

- LLM integration across providers
- Tool orchestration and calling
- Memory management
- Knowledge retrieval (RAG)
- Multi-agent collaboration
- Storage and persistence

What You Can Build

- AI assistants and chatbots
- Autonomous AI workflows
- RAG systems
- Multi-agent pipelines
- Tool-using agents

Why Use Groq?

Groq is an ultra-fast inference platform for open-source LLMs, making it ideal for building responsive AI agents at scale.

Advantages

- Extremely fast inference (lowest latency on market)
- Cheaper than many proprietary APIs
- Supports state-of-the-art open-source models
- Excellent for agentic workflows

Popular Groq Models

Model	Best For
llama3-8b-8192	Fast chatbots, cheap tasks, small agents
llama3-70b-8192	Advanced reasoning, coding, research
deepseek-r1-distill-llama-70b	Deep reasoning, planning, mathematics, autonomous agents

CHAPTER 4

Complete Environment Setup

Step 1 - Install Python

Recommended version: Python 3.11+

```
python --version
```

Step 2 - Create Project Folder

```
mkdir ai_agents  
cd ai_agents
```

Step 3 - Create Virtual Environment

Windows

```
python -m venv venv  
venv\Scripts\activate
```

Linux / Mac

```
python3 -m venv venv  
source venv/bin/activate
```

Step 4 - Install Phidata and Groq

```
pip install phidata groq
```

Step 5 - Install Required Libraries

```
pip install \
```

```
python-dotenv duckduckgo-search \  
fastapi uvicorn streamlit \  
pypdf lancedb sentence-transformers \  
sqlalchemy pandas matplotlib yfinance
```

Step 6 - Configure API Key

Visit <https://console.groq.com> to create your API key, then add it to a .env file:

```
GROQ_API_KEY=your_api_key_here
```

Step 7 - Test Installation

```
from phi.agent import Agent  
from phi.model.groq import Groq  
  
agent = Agent(model=Groq(id='llama3-70b-8192'))  
agent.print_response('Hello')
```

CHAPTER 5

Building Your First Agent

Create a simple agent that can respond to any question using the Groq-powered LLM:

```
from phi.agent import Agent
from phi.model.groq import Groq

agent = Agent(
    model=Groq(id='llama3-70b-8192')
)

agent.print_response(
    'Explain artificial intelligence'
)
```

Workflow

```
Step 1: User sends prompt
Step 2: Phidata forwards prompt to Groq
Step 3: Groq model reasons
Step 4: Response returns to user
```

CHAPTER 6

Agent Components

Every Phidata agent is composed of modular components that you can mix and match:

Agent

- Model (LLM - Groq, OpenAI, etc.)
- Instructions (system prompt and behaviour)
- Tools (functions agent can call)
- Memory (conversation history)
- Knowledge (RAG / vector DB)
- Storage (persistent state)

CHAPTER 7

Instructions and System Prompts

Instructions define the personality, reasoning style, formatting, and behaviour of your agent. Strong instructions produce dramatically better outputs.

Weak Instructions

```
instructions=["Help users"]
```

Strong Instructions

```
instructions=[
    "Act like senior AI engineer",
    "Explain step by step",
    "Always provide examples",
    "Use markdown formatting",
    "Think carefully before answering",
]
```

Professional Agent Example

```
from phi.agent import Agent
from phi.model.groq import Groq

agent = Agent(
    model=Groq(id='llama3-70b-8192'),
    instructions=[
        'Explain step by step',
        'Provide examples',
        'Use markdown',
    ],
    markdown=True,
```


CHAPTER 8

Tools in Phidata

Tools transform agents from passive chatbots into active systems that can take real-world actions.

Calculator Agent

```
from phi.agent import Agent
from phi.tools.calculator import Calculator
from phi.model.groq import Groq

agent = Agent(
    model=Groq(id='llama3-70b-8192'),
    tools=[Calculator()],
    show_tool_calls=True,
)

agent.print_response('Calculate 876 * 45')
```

Web Search Agent

```
from phi.agent import Agent
from phi.tools.duckduckgo import DuckDuckGo
from phi.model.groq import Groq

agent = Agent(
    model=Groq(id='llama3-70b-8192'),
    tools=[DuckDuckGo()],
    show_tool_calls=True,
    markdown=True,
)

agent.print_response('Latest AI news')
```

Python Coding Agent

CHAPTER 9

Memory Systems

Memory allows agents to maintain context across turns in a conversation. Without memory, each message is treated independently.

Basic Memory

```
from phi.agent import Agent
from phi.model.groq import Groq

agent = Agent(
    model=Groq(id='llama3-70b-8192'),
    add_history_to_messages=True,
)

agent.print_response('My name is Saif')
agent.print_response('What is my name?') # Remembers!
```

Advanced Memory Configuration

```
agent = Agent(
    model=Groq(id='llama3-70b-8192'),
    add_history_to_messages=True,
    num_history_responses=15, # remember last 15 exchanges
)
```

■ Set `num_history_responses` based on your use case. More history = better context but higher token usage.

CHAPTER 10

Knowledge Bases

Knowledge bases allow agents to query private documents using semantic vector search.

Install Dependencies

```
pip install pypdf lancedb sentence-transformers
```

PDF Knowledge Base

```
from phi.knowledge.pdf import PDFKnowledgeBase
from phi.vectordb.lancedb import LanceDb

knowledge_base = PDFKnowledgeBase(
    path='documents/manual.pdf',
    vector_db=LanceDb(
        table_name='documents',
        uri='tmp/lancedb',
    ),
)
knowledge_base.load()
```

PDF Assistant Agent

```
agent = Agent(
    model=Groq(id='llama3-70b-8192'),
    knowledge=knowledge_base,
    search_knowledge=True,
    markdown=True,
)
agent.print_response('Summarize chapter 2')
```

CHAPTER 11

RAG Systems

RAG (Retrieval-Augmented Generation) dramatically increases accuracy by grounding responses in your own documents.

RAG Workflow

```
User Question
    |
Vector Database Search
    |
Retrieve Relevant Chunks
    |
Send Context to Groq
    |
Generate Accurate Response
```

Complete RAG Example

```
from phi.agent import Agent
from phi.model.groq import Groq
from phi.knowledge.pdf import PDFKnowledgeBase
from phi.vectordb.lancedb import LanceDb

knowledge_base = PDFKnowledgeBase(
    path='documents',
    vector_db=LanceDb(table_name='knowledge', uri='tmp/lancedb'),
)
knowledge_base.load(recreate=False)

rag_agent = Agent(
    model=Groq(id='llama3-70b-8192'),
    knowledge=knowledge_base,
```


CHAPTER 12

Multi-Agent Systems

Multi-agent systems orchestrate specialized agents that each handle a distinct part of a workflow.

Architecture Example

```
Research Agent -> Analysis Agent -> Writer Agent -> Reviewer Agent
```

Startup Team Example

```
from phi.agent import Agent
from phi.model.groq import Groq

research_agent = Agent(name='Research Agent', role='Researches markets',
                       model=Groq(id='llama3-70b-8192'))
finance_agent = Agent(name='Finance Agent', role='Calculates revenue',
                      model=Groq(id='llama3-70b-8192'))
marketing_agent = Agent(name='Marketing Agent', role='Creates marketing plans',
                        model=Groq(id='llama3-70b-8192'))

startup_team = Agent(
    team=[research_agent, finance_agent, marketing_agent],
    model=Groq(id='llama3-70b-8192'),
    markdown=True,
)

startup_team.print_response('Build AI healthcare startup')
```

CHAPTER 13

Custom Tools

Custom tools connect your agents to any external API, database, or business system.

Weather Tool Example

```
import requests

from phi.tools import Toolkit

class WeatherTools(Toolkit):
    def __init__(self):
        super().__init__(name='weather_tools')
        self.register(self.get_weather)

    def get_weather(self, city: str) -> str:
        response = requests.get(f'https://wttr.in/{city}?format=3')
        return response.text
```

Using the Custom Tool

```
weather_agent = Agent(
    model=Groq(id='llama3-70b-8192'),
    tools=[WeatherTools()],
    show_tool_calls=True,
)

weather_agent.print_response('Weather in Lahore')
```

Autonomous Agents

Autonomous agents plan, reason, and execute multi-step tasks without human intervention at each step.

Autonomous Agent Example

```
from phi.agent import Agent
from phi.model.groq import Groq

agent = Agent(
    model=Groq(id='deepseek-r1-distill-llama-70b'),
    instructions=[
        'Break problems into steps',
        'Think carefully',
        'Validate answers',
        'Act like senior AI architect',
    ],
    markdown=True,
)

agent.print_response('Create AI SaaS roadmap')
```

CHAPTER 15

AI Startup Co-Founder System

Assembles an entire virtual founding team: CEO, researcher, finance analyst, developer, and marketer.

Architecture

```
CEO -> Research Agent -> Finance Agent -> Growth Agent -> Developer Agent -> Pitch Deck Agent
```

Complete Implementation

```
from phi.agent import Agent
from phi.model.groq import Groq
from phi.tools.duckduckgo import DuckDuckGo

research_agent = Agent(name='Research Agent', role='Analyzes competitors',
                        model=Groq(id='llama3-70b-8192'), tools=[DuckDuckGo()])
finance_agent = Agent(name='Finance Agent', role='Calculates business metrics',
                       model=Groq(id='llama3-70b-8192'))
developer_agent = Agent(name='Developer Agent', role='Creates MVP architecture',
                         model=Groq(id='llama3-70b-8192'))
marketing_agent = Agent(name='Marketing Agent', role='Creates growth strategy',
                         model=Groq(id='llama3-70b-8192'))

startup_agent = Agent(
    team=[research_agent, finance_agent, developer_agent, marketing_agent],
    model=Groq(id='llama3-70b-8192'),
    markdown=True,
)

startup_agent.print_response('Build AI fitness startup')
```

CHAPTER 16

FastAPI Integration

Wrap any agent in a production REST API using FastAPI.

```
from fastapi import FastAPI
from pydantic import BaseModel
from phi.agent import Agent
from phi.model.groq import Groq

app = FastAPI()
agent = Agent(model=Groq(id='llama3-70b-8192'))

class RequestData(BaseModel):
    message: str

@app.post('/chat')
def chat(data: RequestData):
    response = agent.run(data.message)
    return {'response': response.content}
```

Run the API

```
uvicorn main:app --reload
```

■ Visit <http://localhost:8000/docs> for the interactive Swagger UI.

CHAPTER 17

Streamlit Frontend

Build a polished web UI for your agent in minutes using Streamlit.

```
import streamlit as st

from phi.agent import Agent

from phi.model.groq import Groq

agent = Agent(model=Groq(id='llama3-70b-8192'))

st.title('AI Assistant')

question = st.text_input('Ask anything')

if st.button('Submit'):
    response = agent.run(question)
    st.write(response.content)
```

Run Streamlit App

```
streamlit run app.py
```

CHAPTER 18

Docker Deployment

Containerise your agent with Docker for reproducible, portable deployments.

Dockerfile

```
FROM python:3.11

WORKDIR /app

COPY requirements.txt .

RUN pip install -r requirements.txt

COPY ..

CMD ["python", "main.py"]
```

Build and Run

```
docker build -t ai-agent .

docker run -p 8000:8000 ai-agent
```

CHAPTER 19

Production Folder Structure

Organise large projects with a clean directory layout that separates concerns and scales with your team.

```
project/
|
+-- agents/
|   +-- research_agent.py
|   +-- coding_agent.py
|   +-- finance_agent.py
|   +-- startup_agent.py
|
+-- tools/
|   +-- weather_tools.py
|   +-- custom_tools.py
|
+-- knowledge/
|   +-- pdfs/
|
+-- api/
|   +-- main.py
|
+-- frontend/
|   +-- app.py
|
+-- .env
+-- requirements.txt
+-- Dockerfile
+-- README.md
```


Best Practices

- **Use clear, detailed instructions** - vague instructions produce vague agents.
- **Use specialised agents** - one focused agent beats one overloaded agent every time.
- **Limit tool access** - only expose tools an agent genuinely needs.
- **Use RAG for accuracy** - ground your agents in real data to eliminate hallucinations.
- **Show tool calls during development** - set `show_tool_calls=True` to debug.
- **Use the right model for the job** - llama3-8b for speed, deepseek-r1 for deep reasoning.
- **Version your knowledge base** - use `recreate=False` in production to avoid re-embedding.
- **Handle errors gracefully** - wrap `agent.run()` calls in `try/except` for production.

Common Errors and Fixes

Error	Fix
GROQ_API_KEY Missing	Add GROQ_API_KEY=your_key to .env and call load_dotenv() at startup.
Missing Tool Dependency	Run: pip install duckduckgo-search (or whichever package the error names)
Slow Responses	Switch to smaller model: Groq(id='llama3-8b-8192')
LanceDB Write Error	Delete tmp/lancedb and re-run knowledge_base.load(recreate=True)
Agent Loops Indefinitely	Set max_iterations on the Agent to cap autonomous loops.

CHAPTER 22

Voice Agents

Voice agents extend AI capabilities to spoken interaction. Combine speech-to-text and text-to-speech around a Phidata + Groq agent core for a complete voice pipeline.

Voice Pipeline Architecture

```
Microphone Input
    |
Whisper (Speech-to-Text)
    |
Phidata + Groq Agent
    |
TTS Engine (gTTS / ElevenLabs)
    |
Speaker Output
```

Install Dependencies

```
pip install openai-whisper gtts pyaudio SpeechRecognition
```

Voice Agent Implementation

```
import speech_recognition as sr
from gtts import gTTS
import os
from phi.agent import Agent
from phi.model.groq import Groq

agent = Agent(
    model=Groq(id='llama3-70b-8192'),
    instructions=['Respond concisely for voice output'],
```

```

)

def listen() -> str:
    recognizer = sr.Recognizer()
    with sr.Microphone() as source:
        print('Listening...')
        audio = recognizer.listen(source)
    return recognizer.recognize_google(audio)

def speak(text: str):
    tts = gTTS(text=text, lang='en')
    tts.save('response.mp3')
    os.system('mpg321 response.mp3')

while True:
    user_input = listen()
    print(f'You: {user_input}')
    response = agent.run(user_input)
    speak(response.content)

```

Streaming Voice with ElevenLabs

```

from elevenlabs.client import ElevenLabs
from elevenlabs import stream

el_client = ElevenLabs(api_key='your_key')

def speak_elevenlabs(text: str):
    audio_stream = el_client.generate(
        text=text,
        voice='Rachel',
        model='eleven_turbo_v2',
        stream=True
    )
    stream(audio_stream)

```

■ *ElevenLabs produces the most natural-sounding voices for production voice agents.*

Finance and Stock Market Agent

Build a financial research agent that fetches live market data, analyses trends, and generates investment summaries autonomously.

Install Dependencies

```
pip install yfinance pandas matplotlib
```

Stock Analysis Tool

```
import yfinance as yf

from phi.tools import Toolkit

class FinanceTools(Toolkit):
    def __init__(self):
        super().__init__(name='finance_tools')
        self.register(self.get_stock_price)
        self.register(self.get_stock_summary)
        self.register(self.get_news)

    def get_stock_price(self, ticker: str) -> str:
        stock = yf.Ticker(ticker)
        hist = stock.history(period='1d')
        price = hist['Close'].iloc[-1]
        return f'{ticker} current price: ${price:.2f}'

    def get_stock_summary(self, ticker: str) -> str:
        stock = yf.Ticker(ticker)
        info = stock.info
        return (
            f"Company: {info.get('longName', 'N/A')}\n"
            f"Market Cap: {info.get('marketCap', 'N/A')}\n"
            f"P/E Ratio: {info.get('trailingPE', 'N/A')}\n"
```

```

        f"52w High:    {info.get('fiftyTwoWeekHigh', 'N/A')}\n"
        f"52w Low:     {info.get('fiftyTwoWeekLow', 'N/A')}"
    )

def get_news(self, ticker: str) -> str:
    stock = yf.Ticker(ticker)
    news = stock.news[:5]
    return '\n'.join([n['title'] for n in news])

```

Finance Agent

```

from phi.tools.duckduckgo import DuckDuckGo

finance_agent = Agent(
    model=Groq(id='llama3-70b-8192'),
    tools=[FinanceTools(), DuckDuckGo()],
    instructions=[
        'Always show current price first',
        'Analyse market trends carefully',
        'Provide clear risk assessment',
        'Use markdown tables for comparison data',
        'Never give financial advice without disclaimers',
    ],
    show_tool_calls=True,
    markdown=True,
)

finance_agent.print_response(
    'Analyse NVIDIA stock and give investment summary'
)

```

Database-Connected Agents

Connect agents to SQL databases so they can query, insert, and analyse structured business data in natural language.

Install Dependencies

```
pip install sqlalchemy psycopg2-binary
```

SQL Tool

```
from sqlalchemy import create_engine, text
from phi.tools import Toolkit

class SQLTools(Toolkit):
    def __init__(self, db_url: str):
        super().__init__(name='sql_tools')
        self.engine = create_engine(db_url)
        self.register(self.run_query)
        self.register(self.list_tables)
        self.register(self.describe_table)

    def list_tables(self) -> str:
        with self.engine.connect() as conn:
            result = conn.execute(
                text("SELECT table_name FROM"
                    " information_schema.tables"
                    " WHERE table_schema='public'"))
            return str([r[0] for r in result])

    def describe_table(self, table: str) -> str:
        with self.engine.connect() as conn:
            result = conn.execute(
                text(f"SELECT column_name, data_type FROM"
```

```

        f" information_schema.columns"
        f" WHERE table_name='{table}''"))
    return str(result.fetchall())

def run_query(self, sql: str) -> str:
    with self.engine.connect() as conn:
        result = conn.execute(text(sql))
        return str(result.fetchall()[:20])

```

Database Agent

```

db_agent = Agent(
    model=Groq(id='llama3-70b-8192'),
    tools=[SQLTools('postgresql://user:pass@localhost/mydb')],
    instructions=[
        'Always list tables first to understand schema',
        'Write safe read-only SELECT queries',
        'Format results as markdown tables',
        'Explain query logic in plain English',
    ],
    markdown=True,
)

db_agent.print_response(
    'Show top 10 customers by revenue this month'
)

```

■ *Always restrict database agents to read-only permissions in production environments.*

Learning Roadmap and Final Projects

Beginner Track

- Python fundamentals, APIs, environment variables
- Build a simple chatbot agent
- Build a calculator agent
- Build a web search agent with DuckDuckGo

Intermediate Track

- Memory management, RAG, custom tools
- Build a PDF RAG assistant for document Q&A;
- Build a coding assistant with PythonTools
- Build a personal research assistant
- Build a voice agent with Whisper + gTTS

Advanced Track

- Multi-agent systems, deployment, scaling, autonomous AI
- Build an AI startup co-founder system
- Build an AI SaaS platform with FastAPI + Streamlit
- Build a finance and market analysis agent
- Build a natural language database query agent
- Deploy everything with Docker to production

Final Projects Checklist

#	Project	Level	Key Tech
1	Chatbot Agent	Beginner	phi.agent + Groq
2	Web Search Agent	Beginner	DuckDuckGo + Groq
3	PDF RAG Assistant	Intermediate	LanceDB + PDFKnowledgeBase

4	Voice Agent	Intermediate	Whisper + gTTS + Groq
5	Finance Analysis Agent	Intermediate	yfinance + custom tools
6	Database Query Agent	Advanced	SQLAlchemy + Groq
7	Multi-Agent Startup Team	Advanced	Multi-agent + DuckDuckGo
8	FastAPI AI Microservice	Advanced	FastAPI + Streamlit
9	Autonomous AI SaaS Platform	Expert	Docker + full stack

You are now equipped to build production-grade AI agents.

Start small. Ship fast. Scale with confidence.

Phidata + Groq - End of Complete Guide - 25 Chapters